

Universitat Politècnica de València
ETSI Aeroespacial y Diseño Industrial

Sistema de Terminación de Vuelo

Flight Termination System



Asignatura: Ingeniería Electrónica
Curso: 2.º Ingeniería Aeroespacial
Autores: Raúl Martínez Martínez, Miguel Vela García y Julen Lorenzo Moa
Fecha: 10 de octubre de 2026

Índice

1. Introducción	3
2. Descripción teórica del sistema FTS	3
3. Simplificación aplicada para la simulación en Proteus	3
4. Diagrama de bloques del hardware	4
5. Circuito electrónico en Proteus	5
6. Diagrama de flujo del funcionamiento	6
7. Descripción de los elementos hardware	8
7.1. Microcontrolador ATmega328P	8
7.2. Sensor PT100	8
7.3. Puente de Wheatstone	8
7.4. Amplificador operacional TL082	8
7.5. Interruptores ARM y FIRE	9
7.6. Puertas lógicas	9
7.7. Optoacopladores PC817	9
7.8. MOSFET y motor DC	9
7.9. Diodo de protección 1N4007	9
7.10. Servomotor	10
7.11. Registros de desplazamiento 4094	10
7.12. Displays de 7 segmentos	10
7.13. ULN2003A	10
7.14. LEDs de estado	10
8. Descripción del código del microcontrolador	11
8.1. Configuración inicial	11
8.2. Definición de umbrales y mapa de pines	11
8.3. Configuración de puertos, ADC y temporizadores	12
8.4. Bucle principal	12
8.5. Interrupción del Timer2	13
8.6. Lectura de temperatura	13
8.7. Control térmico e histéresis	14
8.8. Control proporcional de la compuerta	14
8.9. Estados SAFE, ARMED y FIRE	14
8.10. Visualización mediante registros 4094	14
8.11. Display multiplexado	14
8.12. Resumen del funcionamiento del programa	15
9. Simulación y comprobación del funcionamiento en Proteus	15
9.1. Lectura de temperatura	15
9.2. Control proporcional de la compuerta	16
9.3. Fallo térmico	16
9.4. Estados SAFE, ARMED y FIRE	16
9.5. Limitaciones de la simulación	16

10.Conclusiones	17
11.Bibliografía	17
A. Código completo del microcontrolador	19

1. Introducción

La seguridad operacional constituye uno de los pilares fundamentales en el diseño y desarrollo de sistemas aeroespaciales. En este contexto, el *Flight Termination System* o Sistema de Terminación de Vuelo, conocido como FTS, es un sistema de seguridad cuya función es finalizar el vuelo de un vehículo cuando este se desvía de su trayectoria prevista o supone un riesgo para personas, instalaciones o el entorno.

En un sistema real, el FTS se utiliza principalmente en lanzadores espaciales, cohetes experimentales, misiles y aeronaves no tripuladas. Su activación debe estar muy controlada, ya que un disparo accidental supondría la pérdida de la misión, mientras que un fallo de activación en una situación crítica podría provocar daños graves. Por ello, estos sistemas suelen incorporar redundancia, aislamiento eléctrico, lógica de confirmación y mecanismos de seguridad.

El objetivo de este trabajo es diseñar y simular en Proteus una versión simplificada de un FTS mediante un microcontrolador ATmega328P. La simulación no pretende reproducir todos los detalles de un sistema real, sino implementar sus ideas principales desde el punto de vista de la Ingeniería Electrónica que se ha aprendido durante el curso.

2. Descripción teórica del sistema FTS

Un FTS real está formado por un conjunto de sensores, receptores, unidades de decisión, sistemas de alimentación y actuadores pirotécnicos. Su misión es detectar o recibir una orden de terminación de vuelo y ejecutar una acción irreversible sobre el vehículo.

Desde el punto de vista electrónico, un FTS debe cumplir tres condiciones principales. En primer lugar, debe ser fiable, ya que tiene que responder cuando se produce una situación de emergencia. En segundo lugar, debe ser seguro frente a activaciones accidentales. En tercer lugar, debe estar protegido frente a interferencias, fallos eléctricos y perturbaciones generadas por otros sistemas del vehículo.

Para conseguirlo, se utilizan medidas como señales redundantes, doble confirmación de armado y disparo, aislamiento galvánico, alimentación separada entre control y potencia, protección frente a cargas inductivas y monitorización de variables críticas.

3. Simplificación aplicada para la simulación en Proteus

El sistema real se ha simplificado para poder implementarlo con los conocimientos adquiridos durante el curso. La primera simplificación es que la carga explosiva real se sustituye por un motor DC. Este motor representa visualmente la activación de la carga de terminación, pero no implica ningún proceso destructivo.

La segunda simplificación es la comunicación entre el centro de mando y el vehículo. En un FTS real, la orden podría recibirse inalámbricamente. En este proyecto, la orden se introduce mediante los interruptores físicos ARM y FIRE.

La tercera simplificación afecta al entorno térmico. En un vehículo real habría varios sensores distribuidos en zonas críticas, midiendo la temperatura de la electrónica, la estructura, las baterías o el sistema propulsivo. En nuestro caso se utiliza un único sensor PT100 simulado, acondicionado mediante un puente de Wheatstone y un amplificador operacional.

Además, en un sistema real la apertura de la compuerta de ventilación tendría un efecto físico sobre la temperatura del sistema. Es decir, al abrirse la compuerta aumentaría el flujo de aire, se produciría refrigeración y la temperatura medida por el sensor debería disminuir progresivamente. Ese comportamiento correspondería a un sistema en bucle cerrado, ya que la temperatura controla la apertura del servo y, a su vez, la apertura del servo modifica la temperatura.

En nuestra simulación no se ha modelado ese efecto térmico de realimentación. La temperatura se introduce manualmente mediante el sensor PT100 simulado, y el servomotor actúa únicamente como consecuencia de esa temperatura. Por tanto, el modelo implementado representa la relación directa entre temperatura medida y apertura de la compuerta, pero no contempla la transferencia de calor, la dinámica del flujo de aire ni la evolución temporal de la temperatura tras abrir la compuerta. Esta simplificación se adopta porque modelar correctamente ese comportamiento térmico queda fuera del alcance del trabajo y de los conocimientos necesarios para esta práctica.

La cuarta simplificación corresponde al actuador térmico. El servomotor no actúa sobre un modelo térmico real, sino que representa visualmente la apertura de una compuerta de ventilación. Su ángulo depende de la temperatura medida, cumpliendo así el requisito de que el actuador proporcional dependa de una entrada analógica.

Otra simplificación aplicada al sistema es la gestión temporal de la orden de disparo. En un FTS real, la secuencia de armado y disparo estaría sometida a protocolos estrictos de validación, temporización y seguridad. En nuestro diseño se planteó que, una vez activado el estado ARMED, el operador dispusiera de una ventana temporal limitada, aproximadamente de diez segundos, para pulsar FIRE. Si durante ese intervalo no se recibe la orden de disparo, el sistema vuelve automáticamente al estado SAFE. Esta medida evita que el sistema permanezca armado indefinidamente y reduce el riesgo de activación accidental posterior.

4. Diagrama de bloques del hardware

El sistema puede dividirse en varios bloques funcionales. El primer bloque es la etapa de entrada, formada por los interruptores ARM/FIRE y por el sensor de temperatura PT100. El segundo bloque es el acondicionamiento de señal, formado por el puente de Wheatstone y el amplificador operacional TL082. El tercer bloque es el microcontrolador ATmega328P, que actúa como unidad central de procesamiento.

Finalmente, el sistema cuenta con una etapa de visualización, una etapa de actuación térmica mediante servomotor y una etapa de potencia que activa la carga simulada mediante optoacopladores, puertas lógicas, MOSFET y motor DC.

Bloque	Función dentro del sistema
Entrada de mando	Recoge las órdenes ARM y FIRE mediante interruptores redundantes.
Entrada analógica	Mide la temperatura mediante una PT100 simulada.
Acondicionamiento de señal	Adapta la pequeña señal de la PT100 al rango de lectura del ADC del microcontrolador.
Control principal	El ATmega328P procesa entradas, estados, displays y actuadores.
Visualización	Los displays de 7 segmentos y los LEDs muestran temperatura, apertura de compuerta y estado del sistema.
Actuación térmica	El servomotor abre la compuerta de ventilación de forma proporcional a la temperatura.
Etapas de potencia	Los optoacopladores, la lógica digital y el MOSFET activan el motor que simula la carga del FTS.

Tabla 1: Diagrama de bloques funcional del hardware.

5. Circuito electrónico en Proteus

En la Figura 1 se muestra la captura final del circuito completo implementado en Proteus. En este circuito se integran el microcontrolador ATmega328P, el acondicionamiento de señal del sensor PT100, las entradas de ARM y FIRE, la lógica de redundancia, los optoacopladores, la etapa de potencia con MOSFET, el motor DC que simula la carga de terminación, el servomotor de la compuerta de ventilación y los displays de 7 segmentos.

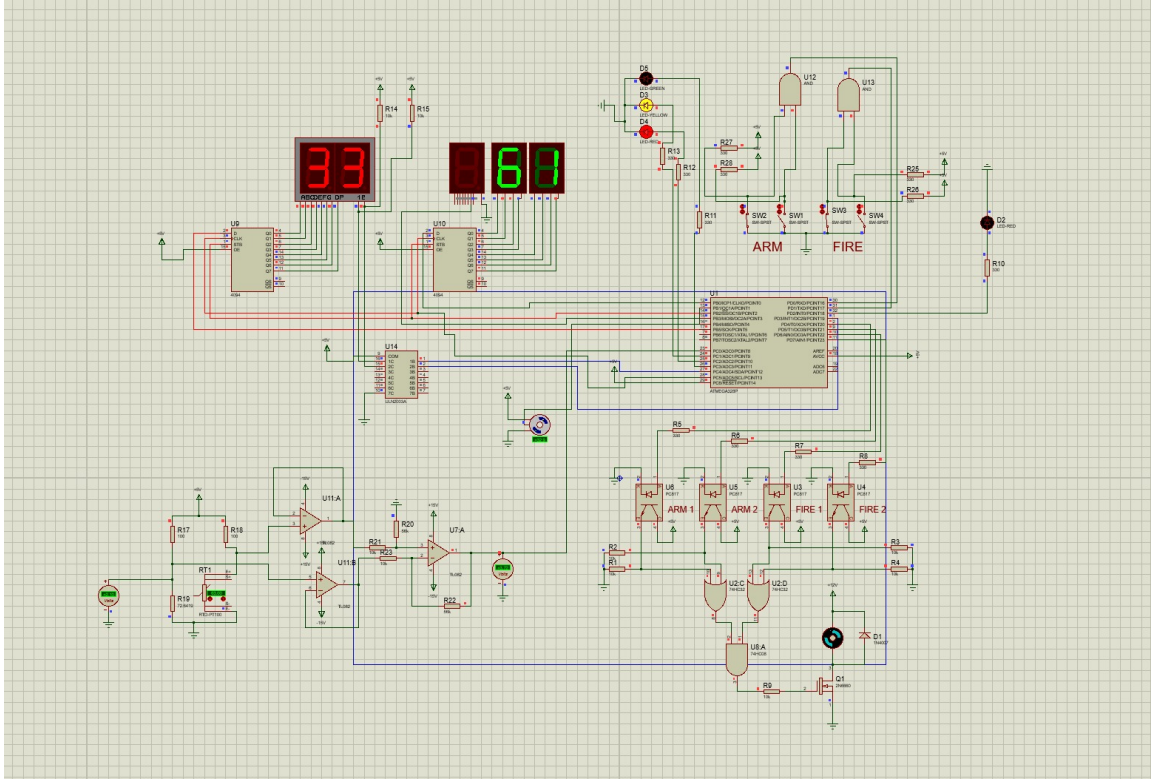


Figura 1: Circuito electrónico completo implementado en Proteus.

6. Diagrama de flujo del funcionamiento

En primer lugar, el sistema mide la temperatura mediante la entrada analógica asociada al sensor PT100. A partir de esta temperatura se calcula la apertura de la compuerta de ventilación. Si la temperatura se encuentra dentro del rango operativo, el sistema permite el armado. Si se supera el límite térmico establecido, se activa el LED de fallo térmico y se bloquea la posibilidad de disparo.

Una vez comprobadas las condiciones térmicas, el sistema espera la orden ARM. Si ARM se activa correctamente, el sistema pasa al estado ARMED. Durante este estado, si se recibe también la orden FIRE dentro de la ventana temporal establecida, el microcontrolador activa las salidas correspondientes y se produce la activación de la carga simulada mediante el motor DC.

Es importante señalar que el diagrama de flujo representa el funcionamiento ideal que se planteó inicialmente para el sistema. En dicho planteamiento se contemplaba que la temperatura se comprobara de forma constante durante todo el proceso, de manera que si durante el estado ARMED o FIRE se superaba el límite térmico, el sistema volviese inmediatamente a SAFE y bloquease el disparo. Sin embargo, en la simulación final de Proteus no se ha implementado un recheck continuo completo de la temperatura en todos los estados, sino una comprobación periódica y simplificada mediante el microcontrolador. Esta decisión se debe a la complejidad añadida que suponía coordinar simultáneamente la lectura analógica, el refresco de los displays multiplexados, el control del servomotor y la lógica ARM/FIRE.

Del mismo modo, en el diagrama de flujo también se planteó una ventana temporal de seguridad: tras activar ARM, el operador dispondría de un tiempo limitado, aproxima-

damente diez segundos, para pulsar FIRE. Si no se recibía la orden FIRE dentro de ese intervalo, el sistema debía volver automáticamente al estado SAFE. En la implementación final esta temporización se ha simplificado. Por tanto, el diagrama de flujo debe interpretarse como la lógica funcional del sistema, mientras que el circuito de Proteus representa una versión simplificada.

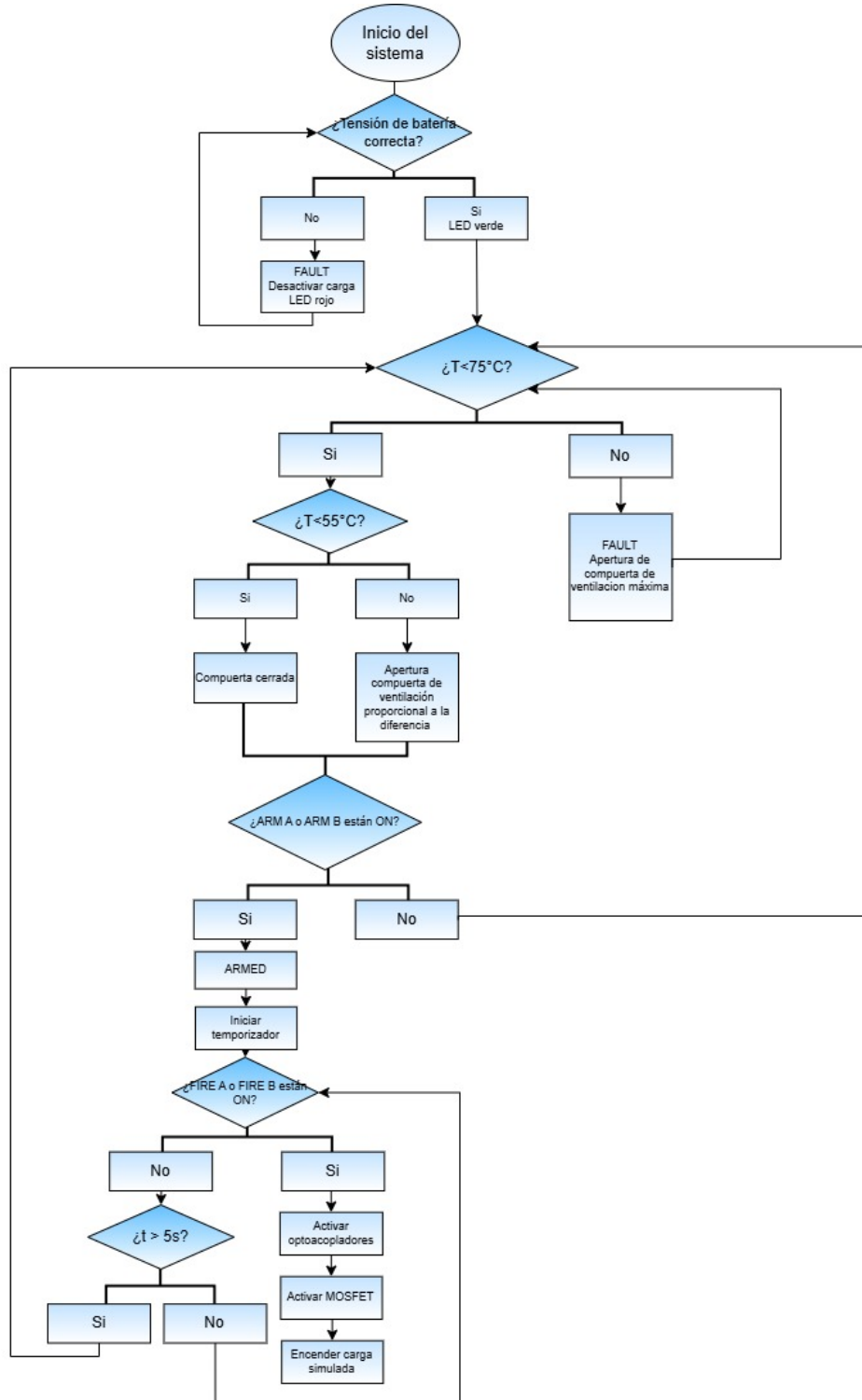


Figura 2: Diagrama de flujo del sistema FTS implementado.

7. Descripción de los elementos hardware

En este apartado se describen los principales componentes empleados en el circuito electrónico. Cada uno de ellos cumple una función concreta dentro del sistema, ya sea como elemento de entrada, acondicionamiento, control, visualización, aislamiento o actuación.

7.1. Microcontrolador ATmega328P

El ATmega328P actúa como controlador principal del sistema. Se encarga de leer la señal analógica procedente del acondicionamiento del sensor PT100, procesar las señales digitales de ARM y FIRE, activar los LEDs de estado, controlar el servomotor, refrescar los displays y generar las salidas hacia la etapa de optoacopladores.

Este microcontrolador resulta adecuado para la simulación porque dispone de conversor analógico-digital, temporizadores internos y puertos digitales suficientes para implementar la lógica del sistema. En el proyecto se programa mediante registros, configurando directamente los puertos, el ADC y los temporizadores.

7.2. Sensor PT100

La PT100 es un sensor resistivo de temperatura. Su resistencia nominal es de aproximadamente $100, \Omega$ a $0,^{\circ}\text{C}$ y aumenta al incrementarse la temperatura. En el sistema simulado se utiliza como entrada analógica principal para representar la temperatura del sistema electrónico.

Como la variación de resistencia de la PT100 es pequeña, no se conecta directamente al ADC del microcontrolador. En su lugar, se utiliza una etapa de acondicionamiento formada por un puente de Wheatstone y un amplificador operacional.

7.3. Puente de Wheatstone

El puente de Wheatstone permite transformar la variación de resistencia de la PT100 en una variación de tensión. Su utilidad principal es comparar la rama donde se encuentra la PT100 con una rama de referencia. De esta forma, se obtiene una señal diferencial asociada al cambio de temperatura.

Este montaje permite reducir la componente continua de la señal y quedarse con una variación más útil para su posterior amplificación. En la práctica, el puente facilita que el microcontrolador pueda interpretar los cambios de temperatura de forma más precisa.

7.4. Amplificador operacional TL082

La señal procedente del puente de Wheatstone es demasiado pequeña para ser leída directamente con buena resolución por el ADC del ATmega328P. Por ello, se utiliza un amplificador operacional TL082 para aumentar la amplitud de la señal y adaptarla al rango de trabajo del microcontrolador.

El objetivo de esta etapa es que la tensión entregada al ADC se mantenga dentro del rango aproximado de 0 a 5, V, aprovechando mejor la resolución de conversión del microcontrolador.

7.5. Interruptores ARM y FIRE

El sistema cuenta con interruptores para las órdenes de ARM y FIRE. La orden ARM representa el armado del sistema, mientras que FIRE representa la autorización final de activación.

Para evitar activaciones accidentales, el sistema exige una secuencia lógica: primero debe activarse ARM y después FIRE. Si no se cumple esta condición, el sistema permanece en estado SAFE y no activa la carga simulada.

7.6. Puertas lógicas

Las puertas lógicas se utilizan para combinar las señales redundantes del sistema. Las puertas OR permiten agrupar señales equivalentes, de forma que si una de ellas está activa, se considera válida la orden correspondiente. La puerta AND final exige que estén presentes simultáneamente las señales de ARM y FIRE antes de activar la etapa de potencia.

Esta estructura permite representar una lógica de seguridad básica, en la que no basta con una única señal aislada para activar el sistema de terminación.

7.7. Optoacopladores PC817

Los optoacopladores PC817 se emplean para aislar eléctricamente el microcontrolador de la etapa de potencia. Internamente, transmiten la señal mediante luz, por lo que no existe una conexión eléctrica directa entre la parte de control y la parte de salida.

Este aislamiento protege al ATmega328P frente a posibles perturbaciones, picos de tensión o interferencias generadas en la zona de potencia. En el circuito se utilizan optoacopladores en las líneas de ARM y FIRE para representar una arquitectura más segura y robusta.

7.8. MOSFET y motor DC

El MOSFET actúa como interruptor electrónico de potencia. Cuando la lógica final confirma que existen señales válidas de ARM y FIRE, el MOSFET conduce y permite el paso de corriente hacia el motor DC.

El motor DC sustituye a la carga explosiva real de un FTS. En la simulación, su giro representa visualmente que la orden de terminación de vuelo se ha ejecutado. Esta sustitución es necesaria por seguridad y por las limitaciones propias de una simulación educativa.

7.9. Diodo de protección 1N4007

El motor DC es una carga inductiva. Cuando se desconecta, puede generar una sobretensión inversa debido al efecto de autoinducción. Para proteger el MOSFET frente a ese pico de tensión, se coloca un diodo 1N4007 en antiparalelo con el motor.

Este diodo proporciona un camino de descarga para la corriente inducida y evita que la sobretensión dañe la etapa de potencia.

7.10. Servomotor

El servomotor representa la compuerta de ventilación del sistema. Su posición depende de la temperatura medida por la PT100. Para temperaturas bajas, la compuerta permanece cerrada. A partir de una temperatura determinada, la apertura aumenta de forma proporcional.

De esta forma se cumple el requisito de que el actuador esté controlado proporcionalmente por una entrada analógica.

7.11. Registros de desplazamiento 4094

Los registros de desplazamiento 4094 permiten controlar displays de 7 segmentos utilizando menos pines del microcontrolador. El ATmega328P envía los datos en serie y el 4094 los convierte en salidas paralelas para activar los segmentos correspondientes.

En el sistema se utilizan para visualizar la temperatura y el porcentaje de apertura de la compuerta. Esta solución permite reducir el número de conexiones necesarias y simplificar el control desde el microcontrolador.

7.12. Displays de 7 segmentos

Los displays de 7 segmentos se utilizan como herramienta de visualización simple. Uno de los bloques muestra la temperatura medida, incluyendo el signo negativo cuando corresponde. El display multiplexado muestra el porcentaje de apertura de la compuerta de ventilación, entre 0 y 99.

El multiplexado consiste en activar alternativamente cada dígito a gran velocidad, de forma que visualmente parece que ambos están encendidos al mismo tiempo, aunque realmente se refrescan de manera secuencial.

7.13. ULN2003A

El ULN2003A se utiliza como etapa de conmutación para seleccionar los dígitos del display multiplexado. Sus salidas son de colector abierto, por lo que funcionan como interruptores a masa.

Cuando una entrada del ULN2003A recibe un nivel alto desde el microcontrolador, la salida correspondiente conecta el común del display a GND y el dígito se enciende. Cuando la entrada está a nivel bajo, la salida queda abierta y el dígito se apaga mediante la resistencia de pull-up conectada a +5V.

7.14. LEDs de estado

El sistema incorpora LEDs para indicar los estados principales. El LED SAFE indica que el sistema está inactivo y seguro. El LED ARMED indica que el sistema está armado, pero aún no ha ejecutado FIRE. El LED FIRE indica que se ha activado la orden de disparo.

Además, se incluye un LED de fallo térmico. Este LED se enciende cuando la temperatura supera el límite de seguridad definido en el programa, avisando de que el sistema se encuentra fuera del rango operativo.

8. Descripción del código del microcontrolador

El código del microcontrolador se ha estructurado por bloques para facilitar su comprensión. En esta sección no se incluye el programa completo, ya que este se incorpora al final del documento como anexo. Aquí se explica la función de cada parte principal del código y su relación con el circuito implementado en Proteus.

8.1. Configuración inicial

En primer lugar, se define la frecuencia de trabajo del microcontrolador:

```
1 #ifndef F_CPU
2 #define F_CPU 16000000UL
3 #endif
```

Esta frecuencia es necesaria para que las funciones de retardo y los temporizadores trabajen correctamente. Posteriormente se incluyen las librerías necesarias para trabajar con registros AVR, interrupciones, retardos, operaciones matemáticas y funciones auxiliares.

Después se declaran las variables globales del sistema. Las variables *VTemp*, *Vout*, *Vref*, *Vpt*, *RxT* y *Treal* se utilizan para transformar la lectura analógica del sensor PT100 en una temperatura real. La variable *T* almacena la temperatura en forma entera para poder mostrarla en los displays.

También se declaran variables de estado:

- *armready*: indica que el sistema está armado.
- *fireready*: indica que se ha autorizado la señal de disparo.
- *tempready*: indica que la temperatura está dentro del rango operativo.

8.2. Definición de umbrales y mapa de pines

A continuación se definen las constantes asociadas al sensor PT100 y a los límites de temperatura. La compuerta comienza a abrirse a partir de 55 °C y alcanza su apertura máxima a 75 °C. Este último valor también se utiliza como límite de seguridad térmica.

```
1 #define TEMP_INICIO_REFRIGERACION 55.0
2 #define TEMP_LIMITE_SERVO 75.0
3 #define TEMP_LIMITE_SEGURIDAD 75.0
4 #define TEMP_MINIMA_SEGURIDAD -40.0
```

El mapa de pines se define mediante directivas *#define*. Esto permite que el código sea más claro, ya que en lugar de trabajar directamente con nombres como *PB0*, *PD4* o *PC5*, se usan nombres relacionados con la función real de cada pin.

Por ejemplo, en el puerto D se encuentran las entradas *ARM* y *FIRE*, el LED de fallo térmico y las salidas hacia los optoacopladores:

```
1 #define ARM_IN PD0
2 #define FIRE_IN PD1
3 #define LED_FALLO_TERM PD2
4 #define DIG2 PD3
5 #define OPTO_ARM1 PD4
6 #define OPTO_ARM2 PD5
```

```

7 #define OPTO_FIRE1          PD6
8 #define OPTO_FIRE2          PD7

```

8.3. Configuración de puertos, ADC y temporizadores

La función `setup()` configura todos los periféricos principales del microcontrolador. Los registros `DDRB`, `DDRC` y `DDRD` determinan qué pines se comportan como entradas y cuáles como salidas.

Las entradas `ARM` y `FIRE` se configuran con resistencias internas de pull-up. Por este motivo, estas entradas son activas en nivel bajo: cuando el interruptor se acciona, el microcontrolador lee un 0 lógico.

```

1 PORTD = (1 << ARM_IN) | (1 << FIRE_IN);

```

El ADC se configura con referencia `AVCC` y prescaler 128:

```

1 ADCSRA = 0b10000111;
2 ADMUX   = 0b01000000;

```

La referencia `AVCC` permite trabajar en el rango de 0 a 5 V. El prescaler 128 se utiliza para reducir la frecuencia del ADC y conseguir una lectura estable de la señal analógica.

El `Timer1` se emplea para generar la señal PWM del servomotor. Se configura con un periodo aproximado de 20 ms, adecuado para el control de servos:

```

1 TCCR1A = (1 << COM1A1);
2 TCCR1B = (1 << WGM13) | (1 << CS11);
3 ICR1    = 20000;

```

El `Timer2` se utiliza como temporizador auxiliar para activar una bandera de actualización aproximadamente cada 200 ms. Esta bandera permite actualizar periódicamente la lectura de temperatura sin bloquear continuamente el programa principal.

8.4. Bucle principal

El bucle principal `loop()` gestiona la secuencia de funcionamiento del sistema. En primer lugar, se actualiza periódicamente el sistema si la bandera `actualizar200ms` está activa. Además, en cada iteración se llama a la función de visualización para refrescar los displays.

La secuencia comienza cuando el operador activa la señal `ARM`. Si la temperatura está dentro del rango operativo, el sistema pasa al estado `ARMED`. En ese momento se activan las dos salidas `ARM` hacia los optoacopladores y se enciende el LED correspondiente.

Durante el estado `ARMED`, el programa comprueba de forma repetida el estado de la temperatura, de la entrada `ARM` y de la entrada `FIRE`. Si `ARM` se desactiva o la temperatura deja de estar dentro del rango permitido, la secuencia se aborta y el sistema vuelve a `SAFE`. Si `FIRE` se activa correctamente mientras el sistema está armado, se activan las dos salidas `FIRE` durante un intervalo breve.

Después de la activación `FIRE`, o si se produce un aborto por temperatura o por desactivación de `ARM`, el sistema desactiva las salidas `ARM` y `FIRE` y vuelve al estado `SAFE`. De esta forma, el microcontrolador mantiene una secuencia básica de seguridad: primero `ARM`, después `FIRE`, y siempre condicionada a que la temperatura sea válida.

```

1 if ((PIND & (1 << ARM_IN)) != 0) {
2     bloqueo_secuencia = 0;
3 }

```

La secuencia de lanzamiento solo se inicia si ARM está activo y no existe bloqueo. Además, el código comprueba que FIRE no estuviera ya activado antes de ARM. Si FIRE está activado de forma prematura, la secuencia se bloquea y se aborta esa vuelta del programa.

Si la temperatura es correcta, el sistema pasa a estado ARMED. En ese momento se activan las dos salidas ARM hacia los optoacopladores y se enciende el LED correspondiente. A partir de ahí, se abre una ventana temporal durante la cual se comprueban tres condiciones: que ARM siga activo, que la temperatura siga siendo válida y que FIRE se pulse correctamente.

Si durante la ventana se baja ARM o aparece un fallo térmico, el sistema aborta la secuencia y vuelve a SAFE. Si FIRE se activa correctamente, se activan las dos salidas FIRE durante un tiempo limitado y después se desactivan.

8.5. Interrupción del Timer2

La interrupción del Timer2 no realiza directamente todos los cálculos del sistema. Su función es contar desbordamientos y activar una bandera cuando se alcanza el tiempo deseado.

```

1 if (contador200ms >= 12) {
2     actualizar200ms = 1;
3     contador200ms = 0;
4 }

```

De esta forma, la interrupción se mantiene corta y el trabajo principal se realiza en el bucle principal. Esto evita que una interrupción demasiado larga interfiera con el refresco de displays o con la lógica ARM/FIRE.

8.6. Lectura de temperatura

La función `leer_temperatura()` se encarga de leer el canal ADC0 y transformar el valor digital obtenido en una temperatura. Primero se inicia la conversión analógica-digital y se espera a que termine:

```

1 ADCSRA |= (1 << ADSC);
2 while ((ADCSRA & (1 << ADSC)) != 0);

```

Después, el valor del ADC se convierte a tensión:

$$V_{out} = 5 \cdot \frac{V_{Temp}}{1023}$$

A partir de esta tensión se calcula la tensión equivalente en el sensor, la resistencia de la PT100 y finalmente la temperatura:

$$T = \frac{R_{xT} - R_0}{R_0 \cdot \alpha}$$

El resultado se guarda en `Treal`, mientras que `T` almacena el valor entero que se muestra en los displays.

8.7. Control térmico e histéresis

La función `actualizar_tempready()` comprueba si la temperatura se encuentra dentro del rango permitido. Si la temperatura supera el límite de seguridad o baja por debajo del mínimo definido, `tempready` pasa a valer 0, se bloquea FIRE y se desactivan las salidas de disparo.

El código incluye histéresis de 2 °C. Esto significa que, cuando el sistema entra en fallo térmico, no vuelve al estado operativo justo al cruzar otra vez el límite, sino que necesita recuperar un pequeño margen de seguridad. Esta medida evita oscilaciones rápidas cuando la temperatura está cerca del umbral.

8.8. Control proporcional de la compuerta

La función `calcular_apertura_y_servo()` calcula la apertura de la compuerta de ventilación. Para temperaturas iguales o menores que 55 °C, la apertura es 0. Entre 55 °C y 75 °C, la apertura aumenta linealmente. Para temperaturas iguales o superiores a 75 °C, la apertura se satura al máximo.

$$\text{Apertura} = \begin{cases} 0, & T \leq 55 \\ \frac{T - 55}{75 - 55} \cdot 99, & 55 < T < 75 \\ 99, & T \geq 75 \end{cases}$$

El valor `apertura` se muestra en el display multiplexado, mientras que `servoCmd` se utiliza para mover el servomotor. Posteriormente, la función `pulso_servo()` actualiza el registro OCR1A, modificando el ancho de pulso PWM aplicado al servo.

8.9. Estados SAFE, ARMED y FIRE

Las funciones `estado_safe()`, `estado_armed()` y `estado_fire()` controlan los LEDs de estado del sistema. En SAFE solo permanece encendido el LED verde de seguridad. En ARMED se enciende el LED de armado. En FIRE se encienden los LEDs que indican que el sistema ha ejecutado la orden de disparo.

Las funciones `activar_arm()`, `desactivar_arm()`, `activar_fire()` y `desactivar_fire()` controlan las salidas hacia los optoacopladores. El sistema utiliza dos salidas ARM y dos salidas FIRE para representar una arquitectura redundante.

8.10. Visualización mediante registros 4094

La función `enviar_4094_doble()` envía información simultáneamente a los registros 4094 asociados al display de temperatura y al display de apertura. Se utilizan dos líneas de datos independientes, una para cada grupo de displays, pero comparten el reloj y el strobe.

Cada bit se introduce mediante un flanco de reloj. Al finalizar el envío de los 8 bits, se activa el strobe para transferir los datos a las salidas del registro.

8.11. Display multiplexado

El porcentaje de apertura se muestra en un display multiplexado de dos dígitos. Para ello, la función `display_apertura()` calcula decenas y unidades, envía el patrón corres-

pondiente al registro 4094 y activa alternativamente cada dígito mediante el ULN2003A.

El multiplexado consiste en encender durante unos milisegundos un dígito y después el otro. Aunque eléctricamente no están encendidos al mismo tiempo, la conmutación es suficientemente rápida para que visualmente parezcan permanecer encendidos de forma continua.

La misma función también controla el signo de la temperatura. Si la temperatura es negativa, se activa el pin asociado al signo; si es positiva, se mantiene apagado.

8.12. Resumen del funcionamiento del programa

En conjunto, el código sigue la siguiente lógica:

1. Leer periódicamente la temperatura.
2. Calcular la apertura proporcional de la compuerta.
3. Actualizar el servo y los displays.
4. Comprobar si la temperatura está dentro del rango operativo.
5. Permitir ARM solo si no hay fallo térmico.
6. Permitir FIRE solo si ARM está activo, FIRE se pulsa en el momento correcto y la temperatura sigue siendo válida.
7. Activar la carga simulada durante un intervalo limitado.
8. Volver a SAFE tras la activación FIRE o tras un aborto por temperatura.

9. Simulación y comprobación del funcionamiento en Proteus

Una vez implementado el circuito y cargado el código en el microcontrolador ATmega328P, se realizó la simulación completa en Proteus. El objetivo principal de esta simulación fue comprobar que los distintos bloques del sistema funcionaban de forma coordinada: lectura de temperatura, visualización, control de la compuerta, lógica ARM/FIRE, bloqueo por temperatura y activación de la carga simulada.

9.1. Lectura de temperatura

La temperatura del sistema se introduce manualmente mediante el sensor PT100 simulado. Al modificar su valor, cambia la tensión de salida del circuito de acondicionamiento. Esta tensión llega a la entrada analógica del microcontrolador, donde se convierte mediante el ADC.

Durante la simulación se comprobó que el valor de temperatura mostrado en los displays variaba correctamente al modificar la PT100. También se verificó que el signo negativo funcionaba cuando la temperatura calculada era inferior a cero.

9.2. Control proporcional de la compuerta

El servomotor representa la apertura de una compuerta de ventilación. En la simulación se comprobó que, para temperaturas bajas, la compuerta permanecía cerrada. A partir de 55,° C, el valor de apertura empezaba a aumentar de forma proporcional.

La apertura se muestra en el display multiplexado como un porcentaje entre 0 y 99. Cuando la temperatura se aproxima al límite superior definido, el display llega al valor máximo y el servomotor alcanza su posición de apertura máxima.

9.3. Fallo térmico

El sistema incorpora una condición de seguridad térmica. Si la temperatura se encuentra fuera del rango operativo, la variable interna `tempready` pasa a valer 0. En ese caso, se enciende el LED de fallo térmico y se bloquea la posibilidad de activar FIRE.

En la simulación se comprobó que, al superar el límite de temperatura, el LED de fallo térmico se encendía como debía. Además, se implementó una pequeña histéresis para evitar que el sistema cambiase rápidamente entre estado válido y fallo cuando la temperatura se encontraba cerca del límite.

9.4. Estados SAFE, ARMED y FIRE

El sistema parte inicialmente del estado SAFE. En este estado, las salidas hacia los optoacopladores permanecen desactivadas y la carga simulada no puede encenderse.

Cuando el operador activa ARM y la temperatura es correcta, el sistema pasa al estado ARMED. En este estado se activan las salidas ARM hacia los optoacopladores y se enciende el LED correspondiente.

Si durante la ventana temporal permitida se activa FIRE, el sistema pasa al estado FIRE. En ese momento se activan las salidas FIRE y la lógica digital permite que el MOSFET active el motor DC. Este motor representa la carga de terminación del sistema.

9.5. Limitaciones de la simulación

Aunque el sistema funciona correctamente a nivel conceptual, la simulación presenta varias limitaciones. La principal es que no se modela el comportamiento térmico real de la compuerta. Es decir, la apertura del servomotor no provoca una reducción automática de la temperatura simulada. La temperatura se introduce manualmente mediante la PT100, por lo que el sistema representa una relación directa entre temperatura medida y apertura, pero no un modelo térmico completo en bucle cerrado.

Además, las temporizaciones implementadas son aproximadas. El programa debe atender simultáneamente la lectura de temperatura, la visualización multiplexada, el control del servo y la lógica ARM/FIRE. Por tanto, los tiempos usados en la simulación son suficientes para representar el funcionamiento del sistema, pero no deben interpretarse como temporizaciones certificadas de un FTS real.

Otra limitación observada durante el desarrollo está relacionada con el uso de interruptores mantenidos para las señales ARM y FIRE. En un sistema real, estas señales estarían gestionadas mediante una lógica de mando más robusta, con enclavamientos, confirmaciones temporales y detección clara de flancos. En la simulación realizada, la secuencia ARM/FIRE se ha representado de forma simplificada, por lo que el objetivo principal es

mostrar la necesidad de una activación secuencial, no reproducir un sistema certificado de enclavamiento.

Finalmente, la carga explosiva real se ha sustituido por un motor DC, lo que permite visualizar la activación del sistema de forma segura dentro de Proteus.

10. Conclusiones

La realización de este proyecto nos ha permitido entender mejor cómo se integran distintos bloques electrónicos dentro de un mismo sistema de control. Aunque el sistema FTS implementado es una versión simplificada de uno real, el trabajo nos ha servido para relacionar conceptos vistos en la asignatura, como entradas analógicas, acondicionamiento de señal, temporizadores, registros, optoacopladores, lógica digital y control de actuadores.

Una de las partes más útiles ha sido comprobar que el diseño de un circuito no consiste solo en conectar componentes, sino en hacer que hardware y código funcionen de forma conjunta. Durante el desarrollo aparecieron problemas con el servomotor, el multiplexado de displays y la selección de los dígitos, lo que nos obligó a analizar señales, revisar conexiones y entender mejor el funcionamiento real de cada bloque.

También hemos aprendido la importancia de simplificar correctamente un sistema físico para poder simularlo. En nuestro caso, la compuerta de refrigeración responde a la temperatura, pero no hemos modelado el efecto real de enfriamiento que tendría sobre el sistema. Esto nos ha hecho ver la diferencia entre una simulación funcional en Proteus y un sistema real en bucle cerrado.

En conclusión, el proyecto nos ha ayudado a aplicar de forma práctica los conocimientos de la asignatura y a comprender mejor la relación entre sensores, microcontrolador, visualización y actuadores. Además, nos ha mostrado la importancia de depurar paso a paso, comprobar cada bloque por separado y justificar las simplificaciones realizadas.

11. Bibliografía

Referencias

- [1] Microchip Technology. (2020). *ATmega328P 8-bit AVR microcontroller data sheet*. Microchip Technology. <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P-Data-Sheet-DS40002061B.pdf>
- [2] Texas Instruments. (2024). *TL082 JFET-input operational amplifier data sheet*. Texas Instruments. <https://www.ti.com/lit/ds/symlink/tl082.pdf>
- [3] Texas Instruments. (2024). *ULN2003A Darlington transistor array data sheet*. Texas Instruments. <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>
- [4] Vishay. (2024). *PC817 optocoupler data sheet*. Vishay Intertechnology. <https://www.vishay.com/docs/83522/pc817.pdf>
- [5] Vishay. (2024). *1N4001-1N4007 general purpose rectifier data sheet*. Vishay Intertechnology. <https://www.vishay.com/docs/88503/1n4001.pdf>
- [6] Labcenter Electronics. (s. f.). *Proteus Design Suite*. Labcenter Electronics. <https://www.labcenter.com/>

- [7] National Aeronautics and Space Administration. (2010). *Range safety user requirements manual*. NASA. <https://www.nasa.gov/>
- [8] National Aeronautics and Space Administration. (s. f.). *Flight termination system and range safety*. NASA. <https://www.nasa.gov/>

A. Código completo del microcontrolador

En este anexo se incluye el código completo cargado en el microcontrolador ATmega328P para la simulación del sistema en Proteus. El programa contiene la configuración de puertos, lectura del sensor PT100, control térmico, gestión de la secuencia ARM/FIRE, control del servomotor y visualización mediante registros 4094.

```
1 #ifndef F_CPU
2 #define F_CPU 16000000UL
3 #endif
4
5 #include <avr/io.h>
6 #include <avr/interrupt.h>
7 #include <util/delay_basic.h>
8 #include <stdlib.h>
9 #include <math.h>
10
11 /* =====
12 1. VARIABLES GENERALES
13 =====
14 */
15 unsigned int VTemp;
16 float Vout, Vref, Vpt, RxT, Treal = 0.0;
17 int T = 0;
18 int apertura = 0; // apertura
19 // mostrada en display, 0-99 %
20 int servoCmd = 0; // mando del servo
21 // , 0-100 %
22 uint8_t armready = 0;
23 uint8_t fireready = 0;
24 uint8_t tempready = 0;
25 volatile uint8_t actualizar200ms = 0;
26
27 /* =====
28 2. CONSTANTES DEL PT100 Y UMBRALES
29 =====
30 */
31 float R17 = 100.0; // Ohm, rama
32 // superior referencia
33 float R18 = 100.0; // Ohm, rama
34 // superior PT100
35 float R19 = 72.6419; // Ohm, rama
36 // inferior referencia
37 float R20 = 56.0; // kOhm, ganancia
38 // diferencial
39 float R21 = 10.0; // kOhm, ganancia
40 // diferencial
41 float R0 = 100.0; // Ohm, PT100 a 0
```

```

C
37 float alpha = 0.0039083;           // coeficiente
   PT100
38
39 #define PT100_SIGN 1.0              // cambiar a -1.0
   si la temperatura va invertida
40
41 #define TEMP_INICIO_REFRIGERACION 55.0 // desde aqu
   empieza a abrir la compuerta
42 #define TEMP_LIMITE_SERVO          75.0 // a esta
   temperatura servo al m ximo
43 #define TEMP_LIMITE_SEGURIDAD      75.0 // a esta
   temperatura se bloquea FIRE
44 #define TEMP_MINIMA_SEGURIDAD      -40.0 // l mite
   inferior operativo
45
46 /* =====
47 3. MAPA DE PINES
48 =====
   */
49
50 /* PORTC */
51 #define LED_ARMED          PC1          // LED estado
   ARMED
52 #define LED_FIRE          PC2          // LED estado FIRE
53 #define LED_SAFE          PC3          // LED estado SAFE
54 #define DIG1              PC4          // selecci n
   d gito 1 del LEDMPX
55 #define CLK_4094          PC5          // reloj com n de
   los 4094
56
57 /* PORTD */
58 #define ARM_IN            PD0          // entrada ARM,
   activa en LOW
59 #define FIRE_IN          PD1          // entrada FIRE,
   activa en LOW
60 #define LED_FALLO_TERM    PD2          // LED fallo
   t rmico
61 #define DIG2              PD3          // selecci n
   d gito 2 del LEDMPX
62 #define OPTO_ARM1         PD4          // opto ARM 1
63 #define OPTO_ARM2         PD5          // opto ARM 2
64 #define OPTO_FIRE1        PD6          // opto FIRE 1
65 #define OPTO_FIRE2        PD7          // opto FIRE 2
66
67 /* PORTB */
68 #define DATA_TEMP        PB0          // dato 4094
   temperatura
69 #define SERVO_PWM         PB1          // salida servo
   OC1A
70 #define STB_4094          PB2          // strobe com n

```

```

4094
71 #define SIGN_TEMP          PB3                // signo negativo
    temperatura
72 #define DATA_APERTURA    PB5                // dato 4094
    apertura
73
74 /* =====
75 4. TABLA 7 SEGMENTOS, C TODO COM N
76 =====
    */
77
78 unsigned char tabla7seg[10] = {
79     0b00111111,                // 0
80     0b00000110,                // 1
81     0b01011011,                // 2
82     0b01001111,                // 3
83     0b01100110,                // 4
84     0b01101101,                // 5
85     0b01111101,                // 6
86     0b00000111,                // 7
87     0b01111111,                // 8
88     0b01101111                // 9
89 };
90
91 /* =====
92 5. PROTOTIPOS
93 =====
    */
94
95 void espera_us(unsigned int us);
96 void espera_ms(unsigned int ms);
97
98 void actualizar_sistema(void);
99 void leer_temperatura(void);
100 void actualizar_tempready(void);
101 void calcular_apertura_y_servo(void);
102 void actualizar_led_fallo_termico(void);
103
104 void activar_arm(void);
105 void desactivar_arm(void);
106 void activar_fire(void);
107 void desactivar_fire(void);
108
109 void estado_safe(void);
110 void estado_armed(void);
111 void estado_fire(void);
112
113 void enviar_4094_doble(unsigned char datoTemp, unsigned char
    datoApertura);
114 void display_apertura(int dato);
115 void apagar_digitos(void);

```

```

116 void encender_dig1(void);
117 void encender_dig2(void);
118
119 void pulso_servo(int mando);
120 void servicio_visualizacion(void);
121
122 /* =====
123 6. SETUP
124 =====
125 */
126 void setup() {
127     DDRC = (1 << LED_ARMED) | (1 << LED_FIRE) | (1 << LED_SAFE)
128         |
129         (1 << DIG1) | (1 << CLK_4094); // PC0 ADC; PC1-
130         PC5 salidas usadas
131     PORTC = 0x00; // salidas
132         inicialmente a LOW
133
134     DDRD = (1 << LED_FALLO_TERM) | (1 << DIG2) |
135         (1 << OPTO_ARM1) | (1 << OPTO_ARM2) |
136         (1 << OPTO_FIRE1) | (1 << OPTO_FIRE2); // PD0/PD1
137         entradas; resto salidas
138     PORTD = (1 << ARM_IN) | (1 << FIRE_IN); // pull-up ARM/
139         FIRE, activos en LOW
140
141     DDRB = (1 << DATA_TEMP) | (1 << SERVO_PWM) |
142         (1 << STB_4094) | (1 << SIGN_TEMP) |
143         (1 << DATA_APERTURA); // PB0,PB1,PB2,PB3
144         ,PB5 salidas
145     PORTB = 0x00; // salidas
146         inicialmente a LOW
147
148     ADCSRA = 0b10000111; // ADC ON,
149         prescaler 128
150     ADMUX = 0b01000000; // AVCC como
151         referencia, ADC0
152
153     TCCR1A = (1 << COM1A1); // Timer1 PWM no
154         inversor en OC1A
155     TCCR1B = (1 << WGM13) | (1 << CS11); // modo 8,
156         prescaler 8
157     ICR1 = 20000; // periodo 20 ms
158         aprox. para servo
159
160     TCCR2A = 0x00; // Timer2 modo
161         normal
162     TCCR2B = (1 << CS22) | (1 << CS21) | (1 << CS20); //
163         prescaler 1024
164     TIMSK2 |= (1 << TOIE2); // interrupci n
165         overflow Timer2

```

```

151     desactivar_arm();                // optos ARM
152         apagados
153     desactivar_fire();              // optos FIRE
154         apagados
155     estado_safe();                  // estado inicial
156         SAFE
157     apagar_digitos();              // multiplexado
158         apagado al inicio
159
160     actualizar_sistema();           // primera lectura
161         del sistema
162     pulso_servo(0);                 // servo cerrado
163         al inicio
164
165     sei();                          // habilita
166         interrupciones globales
167 }
168
169 /* =====
170 7. LOOP PRINCIPAL
171 =====
172 */
173
174 void loop() {
175     unsigned int i, j;
176
177     if (actualizar200ms) {          // actualizaci n
178         peri dica
179         actualizar200ms = 0;
180         actualizar_sistema();
181     }
182
183     servicio_visualizacion();       // refresco
184         continuo del display
185
186     if ((PIND & (1 << ARM_IN)) == 0) { // ARM activo,
187         entrada activa en LOW
188         actualizar_sistema();
189
190         if (tempready == 1) {       // solo arma si la
191             temperatura es v lida
192             armready = 1;
193             fireready = 0;
194
195             activar_arm();           // activa optos
196                 ARM
197             estado_armed();          // LED ARMED
198                 encendido
199
200             for (i = 0; i < 10000; i++) { // ventana de 10 s

```

```

    aprox.
188     actualizar_sistema();
189     servicio_visualizacion();
190
191     if ((PIND & (1 << ARM_IN)) != 0) { // si se baja
        ARM, aborta
192         break;
193     }
194
195     if (tempready == 0) { // si aparece
        fallo t rmico , aborta
196         break;
197     }
198
199     if ((tempready == 1) &&
200         (armready == 1) &&
201         ((PIND & (1 << FIRE_IN)) == 0) &&
202         (fireready == 0)) {
203         fireready = 1; // FIRE autorizado
204     }
205
206     if (fireready == 1) {
207         activar_fire(); // activa optos
        FIRE
208         estado_fire(); // LED FIRE
        encendido
209
210         for (j = 0; j < 500; j++) { // FIRE activo
            500 ms aprox.
211             actualizar_sistema();
212             servicio_visualizacion();
213             espera_ms(1);
214         }
215
216         desactivar_fire(); // apaga FIRE
217         fireready = 0;
218         break;
219     }
220
221     espera_ms(1);
222 }
223
224 armready = 0;
225 fireready = 0;
226
227 desactivar_arm(); // apaga ARM
228 desactivar_fire(); // asegura FIRE
    apagado
229 estado_safe(); // vuelve a SAFE
230 }
231 }

```

```

232 }
233
234 /* =====
235 8. TIMER2: FLAG DE ACTUALIZACION CADA ~200 ms
236 =====
237 */
238 ISR(TIMER2_OVF_vect) {
239     static uint8_t contador200ms = 0;
240
241     contador200ms++; // cuenta
242     overflows de Timer2
243
244     if (contador200ms >= 12) { // 12 overflows
245         200 ms
246         actualizar200ms = 1; // pide
247         actualizaci n en loop
248         contador200ms = 0;
249     }
250 }
251
252 /* =====
253 9. RETARDOS
254 =====
255 */
256
257 void espera_us(unsigned int us) {
258     while (us > 0) {
259         _delay_loop_2(4); // aprox. 1 us a
260         16 MHz
261         us--;
262     }
263 }
264
265 void espera_ms(unsigned int ms) {
266     while (ms > 0) {
267         _delay_loop_2(4000); // aprox. 1 ms a
268         16 MHz
269         ms--;
270     }
271 }
272
273 /* =====
274 10. ACTUALIZACION GENERAL DEL SISTEMA
275 =====
276 */
277
278 void actualizar_sistema(void) {
279     leer_temperatura(); // calcula Treal y
280     T
281     actualizar_tempready(); // comprueba rango

```

```

operativo
274  calcular_apertura_y_servo();           // calcula
      apertura y servo
275  actualizar_led_fallo_termico();       // LED fallo
      t rmico
276 }
277
278 /* =====
279 11. LECTURA Y C LCULO DE TEMPERATURA
280 =====
      */
281
282 void leer_temperatura(void) {
283     float G;
284
285     ADMUX = 0b01000000;           // selecciona ADC0
286     ADCSRA |= (1 << ADSC);        // inicia
      conversi n ADC
287     while ((ADCSRA & (1 << ADSC)) != 0); // espera fin de
      conversi n
288
289     VTemp = ADC;                  // lectura ADC
      0-1023
290     Vout = 5.0 * VTemp / 1023.0;   // tensi n del
      acondicionador
291
292     G = R20 / R21;                // ganancia del
      amplificador
293     Vref = 5.0 * R19 / (R17 + R19); // tensi n de
      referencia del puente
294     Vpt = Vref + PT100_SIGN * (Vout / G); // tensi n
      equivalente del PT100
295
296     if (Vpt < 0.01) Vpt = 0.01;   // evita divisi n
      por cero
297     if (Vpt > 4.99) Vpt = 4.99;   // limita al rango
      f sico
298
299     RxT = R18 * Vpt / (5.0 - Vpt); // resistencia
      equivalente PT100
300     Treal = (RxT - R0) / (R0 * alpha); // temperatura
      calculada
301     T = (int)(Treal);              // temperatura
      entera para display
302 }
303
304 /* =====
305 12. CONTROL T RMICO
306 =====
      */
307

```

```

308 void actualizar_tempready(void) {
309     if ((Treal >= TEMP_MINIMA_SEGURIDAD) &&
310         (Treal < TEMP_LIMITE_SEGURIDAD)) {
311         tempready = 1; // temperatura
312         v lida
313     } else {
314         tempready = 0; // temperatura
315         fuera de rango
316         fireready = 0; // bloquea FIRE
317         desactivar_fire(); // asegura FIRE
318         apagado
319     }
320 }
321 void calcular_apertura_y_servo(void) {
322     if (Treal <= TEMP_INICIO_REFRIGERACION) {
323         apertura = 0; // compuerta
324         cerrada
325         servoCmd = 0; // servo m nimo
326     }
327     else if (Treal < TEMP_LIMITE_SERVO) {
328         apertura = (int)((Treal - TEMP_INICIO_REFRIGERACION) *
329             99.0 /
330             (TEMP_LIMITE_SERVO - TEMP_INICIO_REFRIGERACION
331             ));
332         servoCmd = (int)((Treal - TEMP_INICIO_REFRIGERACION) *
333             100.0 /
334             (TEMP_LIMITE_SERVO - TEMP_INICIO_REFRIGERACION
335             ));
336     }
337     else {
338         apertura = 99; // apertura
339         m xima mostrada
340         servoCmd = 100; // servo m ximo
341     }
342     if (apertura < 0) apertura = 0; // saturaci n
343     inferior display
344     if (apertura > 99) apertura = 99; // saturaci n
345     superior display
346     if (servoCmd < 0) servoCmd = 0; // saturaci n
347     inferior servo
348     if (servoCmd > 100) servoCmd = 100; // saturaci n
349     superior servo
350     pulso_servo(servoCmd); // actualiza PWM
351     servo
352 }

```

```

345 void actualizar_led_fallo_termico(void) {
346     if (Treal >= TEMP_LIMITE_SEGURIDAD) {
347         PORTD |= (1 << LED_FALLO_TERM);           // LED fallo
348         t rmico ON
349     } else {
350         PORTD &= ~(1 << LED_FALLO_TERM);           // LED fallo
351         t rmico OFF
352     }
353 }
354
355 /* =====
356 13. ARM / FIRE
357 =====
358 */
359
360 void activar_arm(void) {
361     PORTD |= (1 << OPTO_ARM1) | (1 << OPTO_ARM2); // activa optos
362     ARM
363 }
364
365 void desactivar_arm(void) {
366     PORTD &= ~((1 << OPTO_ARM1) | (1 << OPTO_ARM2)); // apaga
367     optos ARM
368 }
369
370 void activar_fire(void) {
371     PORTD |= (1 << OPTO_FIRE1) | (1 << OPTO_FIRE2); // activa
372     optos FIRE
373 }
374
375 void desactivar_fire(void) {
376     PORTD &= ~((1 << OPTO_FIRE1) | (1 << OPTO_FIRE2)); // apaga
377     optos FIRE
378 }
379
380 /* =====
381 14. LEDS DE ESTADO
382 =====
383 */
384
385 void estado_safe(void) {
386     PORTC &= ~(1 << LED_ARMED);           // ARMED OFF
387     PORTC &= ~(1 << LED_FIRE);           // FIRE OFF
388     PORTC |= (1 << LED_SAFE);           // SAFE ON
389 }
390
391 void estado_armed(void) {
392     PORTC |= (1 << LED_ARMED);           // ARMED ON
393     PORTC &= ~(1 << LED_FIRE);           // FIRE OFF
394     PORTC &= ~(1 << LED_SAFE);           // SAFE OFF
395 }

```

```

388
389 void estado_fire(void) {
390     PORTC |= (1 << LED_ARMED);           // ARMED ON
391     PORTC |= (1 << LED_FIRE);           // FIRE ON
392     PORTC &= ~(1 << LED_SAFE);         // SAFE OFF
393 }
394
395 /* =====
396 15. ENV O A 4094
397 =====
398 */
398
399 void enviar_4094_doble(unsigned char datoTemp, unsigned char
datoApertura) {
400     unsigned char i;
401
402     for (i = 0; i < 8; i++) {
403         if (datoTemp & 0x01) PORTB |= (1 << DATA_TEMP);           //
bit temperatura
404         else PORTB &= ~(1 << DATA_TEMP);
405
406         if (datoApertura & 0x80) PORTB |= (1 << DATA_APERTURA);
// bit apertura
407         else PORTB &= ~(1 << DATA_APERTURA);
408
409         espera_us(2);
410
411         PORTC |= (1 << CLK_4094);           // flanco subida
CLK
412         espera_us(2);
413         PORTC &= ~(1 << CLK_4094);           // flanco bajada
CLK
414         espera_us(2);
415
416         datoTemp = datoTemp / 2;           // temperatura LSB
primero
417         datoApertura = datoApertura * 2;   // apertura MSB
primero
418     }
419
420     PORTB |= (1 << STB_4094);           // STROBE activa
salida
421     espera_us(2);
422     PORTB &= ~(1 << STB_4094);
423     espera_us(2);
424 }
425
426 /* =====
427 16. DISPLAY MULTIPLEXADO DE APERTURA
428 =====
429 */

```

```

429
430 void apagar_digitos(void) {
431     PORTC &= ~(1 << DIG1);           // DIG1 OFF
432     PORTD &= ~(1 << DIG2);           // DIG2 OFF
433 }
434
435 void encender_dig1(void) {
436     PORTC |= (1 << DIG1);             // DIG1 ON
437 }
438
439 void encender_dig2(void) {
440     PORTD |= (1 << DIG2);             // DIG2 ON
441 }
442
443 void display_apertura(int dato) {
444     unsigned char dec, uni, datoTempBCD;
445     int tempAbs;
446
447     if (dato < 0)    dato = 0;         // evita negativos
448     if (dato > 99)   dato = 99;        // limita a 2
449                                     d gitos
450
451     dec = dato / 10;                   // decenas
452     uni = dato % 10;                   // unidades
453
454     if (T < 0) {
455         PORTB |= (1 << SIGN_TEMP);     // signo negativo
456         ON
457         tempAbs = abs(T);
458     } else {
459         PORTB &= ~(1 << SIGN_TEMP);    // signo negativo
460         OFF
461         tempAbs = T;
462     }
463
464     if (tempAbs > 99) tempAbs = 99;     // temperatura
465                                     limitada a 2 d gitos
466
467     datoTempBCD = ((tempAbs / 10) << 4) + (tempAbs % 10); //
468                                     temperatura en BCD
469
470     apagar_digitos();                  // apaga d gitos
471                                     antes de cambiar segmentos
472     espera_us(200);
473
474     enviar_4094_doble(datoTempBCD, tabla7seg[dec]); // carga
475                                     decenas apertura
476     encender_dig1();                   // enciende primer
477                                     d gito
478     espera_ms(4);
479

```

```

472     apagar_digitos(); // apaga antes de
        cambiar al otro
473     espera_us(200);
474
475     enviar_4094_doble(datoTempBCD, tabla7seg[uni]); // carga
        unidades apertura
476     encender_dig2(); // enciende
        segundo d gito
477     espera_ms(4);
478
479     apagar_digitos(); // evita ghosting
480 }
481
482 /* =====
483 17. SERVO
484 =====
        */
485
486 void pulso_servo(int mando) {
487     unsigned int us;
488
489     if (mando < 0) mando = 0; // m nimo servo
490     if (mando > 100) mando = 100; // m ximo servo
491
492     us = 1000 + ((unsigned long)mando * 556) / 100; // pulso
        calibrado del servo
493
494     cli(); // evita escritura
        interrumpida de OCR1A
495     OCR1A = us; // actualiza
        posici n servo
496     sei(); // reactiva
        interrupciones
497 }
498
499 /* =====
500 18. SERVICIO DE VISUALIZACI N
501 =====
        */
502
503 void servicio_visualizacion(void) {
504     display_apertura(apertura); // refresca
        display de apertura
505 }

```

Listing 1: Código completo del microcontrolador ATmega328P.